inbound email and send another email (remember to send it to `support@example.com`).

Now, quit your server and start up the Rails console. This will allow us to check that a new `SupportRequest` was created (remember we have to format this to fit in the book, so your output will be on fewer, longer lines):

```
> bin/rails console
irb(main):001:0> SupportRequest.first
   (1.5ms)  SELECT sqlite_version(*)
  SupportRequest Load (0.1ms)
    SELECT "support_requests".* FROM "support_requests"
      ORDER BY "support_requests"."id" ASC LIMIT ?  [["LIMIT", 1]]
=> #<SupportRequest
       id: 1,
       email: "chris@somewhere.com",
       subject: "Missing book!",
       body: "I can't find my book that I ordered.  Please help!",
       order_id: nil,
       created_at: "2021-01-19 12:29:17",
       updated_at: "2021-01-19 12:29:17">
```

You should see the data you entered into the conductor saved in the `SupportRequest` instance. You can also try this using the email of an order you have in your system to verify it locates the most recent order. Of course, manually checking our code isn't ideal. We would like to have an automated test. Fortunately, Rails provides a simple way to test our mailboxes, which we'll learn about now.

Testing Our Mailbox

When we used the generator to create our mailbox, you probably noticed the file `test/mailboxes/support_mailbox_test.rb` get created. This is where we'll write our test. Since we generally know how to write tests, all we need to know now is how to trigger an email. Action Mailbox provides the method `receive_inbound_email_from_mail` which we can use in our tests to do just that.

We need two tests to cover the functionality of our mailbox. The first is to send an email from a customer without an order and verify we created a